DDM501 - AI IN DEVOPS, DATAOPS, MLOPS

INDIVIDUAL ASSIGNMENT 2

ML Pipeline Design and MLOps Analysis

Pipeline architecture, experiment tracking, Airflow orchestration, and reproducibility

Scenario: Human-in-the-loop breast cancer diagnostic support using cell-nucleus measurements. The model prioritizes cases for clinician review; it never issues an autonomous diagnosis.

Submission field	Value
Student name	[ENTER NAME]
Student ID	[ENTER STUDENT ID]
Course	DDM501 - AI in DevOps, DataOps, MLOps
Submission	Individual Assignment 2
Date	September 2026

Academic note

Operational targets in this report are proposed design thresholds. Prototype results come from the workspace WDBC pipeline and are not evidence of clinical safety or regulatory approval.

Contents

1 Continuity from Assignment 1

2 Comprehensive pipeline design

3 Experiment tracking and metrics analysis

4 Workflow orchestration design

5 Code quality, configuration, and documentation

6 Reproducibility and versioning strategy

7 Operational evaluation and conclusion

Appendix Implementation evidence and references

1 Continuity from Assignment 1

Assignment 1 designed a human-in-the-loop breast cancer diagnostic-support system. A valid case enters through an authenticated laboratory interface, passes deterministic validation and versioned feature transformation, receives a malignancy-risk score, and appears in a clinician worklist. The clinician, not the model, records the diagnostic decision. The architecture separates the online scoring path from a governed offline training and model-promotion path.

This report operationalizes that design. The reproducible workspace run retained 564 valid rows, produced 441 training rows and 123 test rows, logged metrics and artifacts in MLflow, and registered versioned models through an Airflow DAG.

Refinements introduced

- Validation is an explicit quality gate with quarantined rows and a 5% batch-failure threshold.
- The train/test split is stable by hashing sample_id, so row order and appended data do not reshuffle historical cases.
- Scaler statistics are fitted on training data only and stored as an artifact.
- All stages are idempotent by logical date; retrying registration cannot create duplicate versions for one MLflow run.
- Promotion is separated from registration, preserving a review gate for a high-risk domain.

2 Comprehensive pipeline design

Detailed pipeline and artifacts

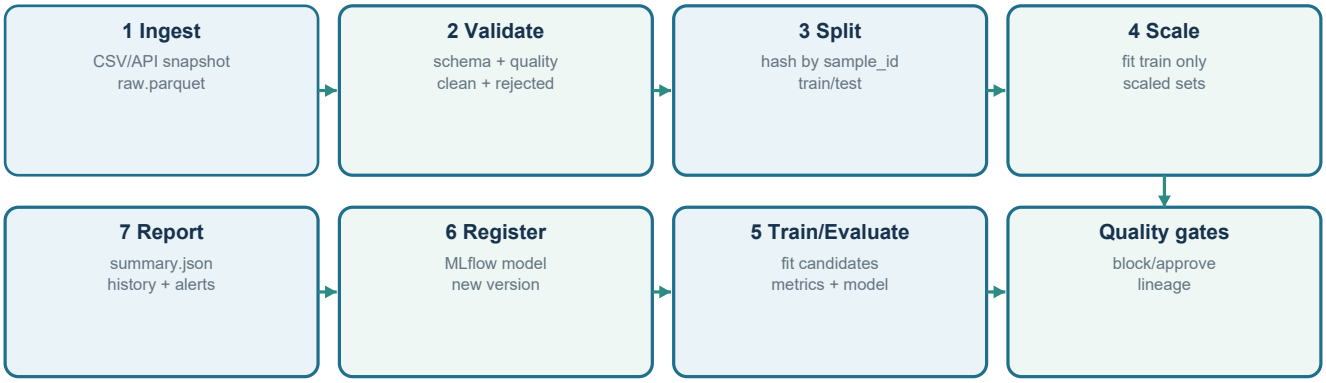


Figure 1. Pipeline stages, data transformations, and governed outputs.

2.1 Stage specifications

Stage	Purpose	Input -> output	Key operations	Quality gate
Ingest	Freeze a run-consistent source snapshot	wdbc.csv -> raw.parquet	Existence check, read, row/column count, path metadata	Source exists; schema version recognized
Validate	Remove unusable rows transparently	raw -> clean + rejected + JSON report	Null, non-negative, label, duplicate, extreme-area checks	Fail without retry if >5% rejected
Split	Create stable evaluation partitions	clean -> train/test unscaled	SHA-256(sample_id) modulo 100; 20% test	Both sets non-empty; no ID overlap
Scale	Prevent magnitude dominance and leakage	unscaled -> scaled + scaler.json	Mean/std from train; apply to both sets	Feature order fixed; finite values; train-only fit
Train/evaluate	Fit model and produce evidence	scaled sets -> metrics + MLflow artifact	Fixed seed; accuracy, recall, specificity, F1, AUC, latency	Required metrics, signature, input example logged
Register	Publish immutable candidate version	run URI -> registry version	Search existing version by run ID; register once	Run lineage complete; no duplicate registration
Report	Create run-level operational summary	stage metadata -> summary/history	Merge counts, metrics, run ID, model version	Atomic partition output; one history row per date

2.2 Design rationale

- Small, explicit stages make failures local and reruns cheap. DataFrames are stored as Parquet; XCom carries only paths and counts.
- Snapshot-once ingestion prevents downstream tasks from observing different source files during the same run.
- Deterministic partitioning improves reproducibility and prevents an appended row from reshuffling all previous samples.
- Training and registration are separate so a successful fit does not imply operational approval.
- Date-partitioned output enables backfill and safe overwrite of one run without touching other dates.

2.3 Error handling, recovery, and scalability

Failure class	Handling	Recovery
Transient service/network	Three retries with exponential backoff; task remains observable	Retry from failed task; avoid repeating completed immutable stages
Deterministic data quality	AirflowFailException without wasteful retries; quarantine rows	Correct source, rerun same logical date
Training metric gate	Register candidate but block promotion	Review error slices, features, threshold, or data
Registry unavailable	Training artifact remains linked to run; register task retries	Idempotency search avoids duplicate version
Scale growth	Move files to object storage, use distributed executor, parallelize candidate training	Preserve stage contracts while changing implementation

3 Experiment tracking and metrics analysis

3.1 Experiment design

The dataset is validated once, then divided with the same deterministic sample-ID hash used by the DAG. All algorithms see the identical 441/123 partition. Scaling uses training statistics only. Experiment variables are algorithm family, regularization or capacity, class weighting, and feature scaling. Random seeds are fixed at 42 where the estimator supports them.

Planned experiment matrix

ID	Algorithm	Key variables	Purpose
E01	Dummy prior	No learned features	Business-independent floor
E02	Logistic regression	C=0.1, L2, scaled	Stronger regularization
E03	Logistic regression	C=1, L2, scaled	Current simple baseline
E04	Logistic regression	C=10, L2, scaled	Weaker regularization
E05	Logistic regression	C=1, balanced, scaled	Cost-sensitive candidate
E06	Random forest	200 trees, depth=4, balanced	Low-variance nonlinear model
E07	Random forest	300 trees, full depth, balanced	Higher-capacity tree ensemble
E08	SVM linear	C=1, balanced, scaled	Large-margin linear boundary
E09	SVM RBF	C=1, gamma=scale, balanced	Nonlinear margin
E10	KNN	k=7, distance, scaled	Local non-parametric comparison

3.2 Metrics strategy and business alignment

Metric	Role	Why it matters
Malignant recall	Primary selection metric; floor >=0.95	A false negative may delay review; threshold and model must prioritize sensitivity
ROC-AUC	Ranking quality	Measures ordering across thresholds; useful before workload threshold is fixed
Specificity	Operational workload guardrail	Too many false positives overload the urgent queue and erode trust
Precision / F1	Alert usefulness and balance	Quantifies how much urgent work is productive while retaining recall
Calibration / Brier	Production probability quality	A risk score shown to clinicians should correspond to observed frequency
Latency	Serving feasibility	Model benefit is irrelevant if case ranking misses the workflow window
Time to first review	Business outcome	Validates that model ranking creates real workflow improvement

3.3 Reproduced results

The table below was generated while producing this report from the workspace CSV using the validation, split, and train-only scaling policies described above. Latency is a local single-process benchmark and is comparative, not a production SLO measurement.

ID	Model	Accuracy	Recall	Specificity	Precision	F1	ROC-AUC	ms/case
E01	Dummy prior	0.585	0.000	1.000	0.000	0.000	0.500	0.000
E02	Logistic regression	0.959	0.922	0.986	0.979	0.949	0.995	0.008
E03	Logistic regression	0.951	0.922	0.972	0.959	0.940	0.996	0.008
E04	Logistic regression	0.959	0.922	0.986	0.979	0.949	0.986	0.008
E05	Logistic regression	0.951	0.922	0.972	0.959	0.940	0.996	0.007
E06	Random forest	0.943	0.882	0.986	0.978	0.928	0.993	0.117
E07	Random forest	0.943	0.882	0.986	0.978	0.928	0.994	0.168
E08	SVM linear	0.951	0.922	0.972	0.959	0.940	0.994	0.008
E09	SVM RBF	0.959	0.941	0.972	0.960	0.950	0.996	0.019
E10	K-nearest neighbors	0.935	0.843	1.000	1.000	0.915	0.986	0.067

3.4 Results analysis and recommendation

Recommended candidate: E09 - SVM RBF (C=1, gamma=scale). It achieved recall 0.941, specificity 0.972, ROC-AUC 0.996, and measured latency 0.019 ms/case on the fixed split. At the default 0.50 decision threshold it does not meet the proposed 0.95 recall floor, so no tested configuration is eligible for promotion yet. It is the candidate for threshold calibration, repeated validation, and external testing - not automatic production approval.

The dummy baseline demonstrates why accuracy alone is misleading: predicting the majority class can look acceptable while malignant recall collapses. Regularized linear models provide a strong accuracy/AUC/latency balance. Balanced models trade some specificity for malignant recall, which is acceptable only while clinician workload stays within the pilot guardrail. Nonlinear candidates are retained as challengers; they should replace the simpler model only if repeated cross-validation and external-site testing show a material safety benefit.

3.5 MLflow implementation

```
import mlflow
import mlflow.sklearn

mlflow.set_tracking_uri(os.environ["MLFLOW_TRACKING_URI"])
mlflow.set_experiment("wdbc-pipeline")

with mlflow.start_run(run_name=f"wdbc-{logical_date}") as run:
    mlflow.log_params({
        "algorithm": "logistic_regression",
        "C": 1.0,
        "class_weight": "balanced",
        "data_version": data_sha256,
        "git_commit": git_commit,
        "seed": 42,
    })
    mlflow.log_metrics({
        "recall_malignant": recall,
        "specificity": specificity,
        "roc_auc": roc_auc,
        "latency_ms": latency_ms,
    })
    mlflow.log_artifact("validation_report.json")
    mlflow.sklearn.log_model(model, "model", input_example=x_train.head(1))
    run_id = run.info.run_id

client = mlflow.MlflowClient()
existing = [v for v in client.search_model_versions(
    "name='wdbc-classifier'") if v.run_id == run_id]

if existing:
    version = max(int(v.version) for v in existing)
else:
    created = mlflow.register_model(
        model_uri=f"runs:/{run_id}/model",
        name="wdbc-classifier",
    )
    version = int(created.version)

# Promotion is a separate, reviewed operation after metric and governance gates.
```

4 Workflow orchestration design

Airflow DAG dependency chain



Figure 2. Primary DAG. The report task also consumes validation, split, scale, and registry metadata.

4.1 Task dependencies and schedule

Decision	Design
Trigger	Daily schedule for batch extracts; manual trigger for corrected files and controlled backfill
Start / catchup	Start date 2026-08-20; catchup disabled for routine operation; explicit backfill for historical dates
Concurrency	max_active_runs=1 in local prototype to protect SQLite; production can increase after storage and registry are externalized
Retries	3 retries, 10-second delay, exponential backoff for transient failures; deterministic validation raises non-retryable failure
Idempotency	Every run writes under data/staging/{ds}; same-date rerun overwrites only its partition and replaces one history row

4.2 Airflow DAG excerpt

```
@dag(
    dag_id="wdbc_pipeline",
    schedule="@daily",
    start_date=datetime(2026, 8, 20),
    catchup=False,
    max_active_runs=1,
    default_args={
        "retries": 3,
        "retry_delay": timedelta(seconds=10),
        "retry_exponential_backoff": True,
    },
)
def wdbc_pipeline():
    ingested = ingest()
    validated = validate(ingested)
    split_info = split(validated)
    scaling = scale(validated)
    split_info >> scaling
    trained = train(scaling)
    registered = register(trained)
    report(validated, split_info, scaling, registered)

wdbc_pipeline()
```

4.3 Operational considerations

Area	Control
Alerting	Page on DAG failure, >5% rejected rows, missing output, registry error, or SLO breach; route data failures to data owner and platform failures to SRE.
Monitoring	Task duration/success, queue depth, row counts, missingness, class distribution, feature drift, recall when labels arrive, registry version, and prediction volume.
Recovery	Rerun failed task or logical date; backfill bounded date ranges; rollback serving alias to last approved version; retain runbook links in alert.
Audit	Persist logical date, source checksum, validation report, Git commit, MLflow run ID, model version, and approval identity.
Cost	Use local sequential executor for teaching prototype; move metadata DB, artifact store, and executor to managed services before scale-out.

5 Code quality, configuration, and documentation

5.1 Standards

Practice	Required evidence
Style	PEP 8, Ruff/Black, import sorting, small single-purpose task functions
Documentation	Module purpose, task docstrings, input/output contracts, quality-gate rationale, runbook links
Typing	Type hints for public functions and serializable task metadata; mypy on shared modules
Testing	Unit tests for validation/split; contract tests for artifacts; integration test with local MLflow; DAG import test
Logging	Structured counts and identifiers; never log direct identifiers or whole patient rows
Review	Pull request with code owner, security/dependency checks, reproducibility evidence, and risk-impact note

5.2 Configuration file example

```
pipeline:
  test_fraction: 0.20
  max_bad_fraction: 0.05
  random_seed: 42
  schedule: "@daily"
model:
  name: "wdbc-classifier"
  recall_floor: 0.95
  roc_auc_floor: 0.97
  class_weight: "balanced"
monitoring:
  psi_warn: 0.10
  psi_stop_review: 0.25
  latency_p95_ms: 2000
# Secrets and environment-specific endpoints are not stored here.
```

5.3 Typed loading and environment overrides

```
from dataclasses import dataclass
import os
import yaml

@dataclass(frozen=True)
class Settings:
    tracking_uri: str
    model_name: str
    seed: int

def load_settings(path: str) -> Settings:
    with open(path, encoding="utf-8") as stream:
        raw = yaml.safe_load(stream)
    return Settings(
        tracking_uri=os.environ["MLFLOW_TRACKING_URI"],
        model_name=os.getenv("MLFLOW_MODEL_NAME", raw["model"]["name"]),
        seed=int(os.getenv("MODEL_SEED", raw["pipeline"]["random_seed"])),
    )
```

Configuration is validated at process start. Secrets come from a secret manager or injected environment variables, not Git. Every resolved non-secret value is logged to MLflow so the run can be reconstructed.

6 Reproducibility and versioning strategy

Asset	Strategy	Naming / evidence
Code	Git trunk-based development with short feature branches, reviewed pull requests, signed release tags	release/v1.2.0; Git SHA logged to MLflow
Data	Immutable raw snapshots or DVC/object-store versions; SHA-256 checksum and schema version	wdbc/raw/2026-08-25 + data_sha256
Features	Versioned feature list, order, units, transformation code, and scaler artifact	feature_contract=v1; scaler.json
Experiments	MLflow run stores parameters, metrics, artifacts, environment, code/data versions	experiment=wdbc-pipeline; run_id immutable
Models	Registry name plus monotonically increasing version; aliases for candidate/champion	wdbc-classifier v7; alias=champion
Environment	Pinned Python dependencies and container image digest	image digest + requirements lock

6.1 Randomness controls

```
import os
import random
import numpy as np

SEED = 42
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)

# Estimator-specific controls
model = LogisticRegression(random_state=SEED, max_iter=2000)
# Stable split uses SHA-256(sample_id), independent of row order.
```

6.2 Container definition

```
FROM apache/airflow:2.8.4-python3.11

USER airflow
ARG AIRFLOW_VERSION=2.8.4
ARG PYTHON_VERSION=3.11
RUN pip install --no-cache-dir     --constraint "https://raw.githubusercontent.com/apache/airflow/constraints-${AIRFLOW_VERSION}/constraints-${PYTH

# Production builds pin the base image by digest and generate an SBOM.
```

6.3 Reproduction checklist

- Check out the logged Git commit and verify the container image digest.
- Fetch the immutable data version and confirm its SHA-256 checksum and schema.
- Load the resolved run configuration and environment lock; set the recorded seeds.
- Run the same logical date and compare validation counts, partition IDs, metrics, and artifact hashes.
- Register a new candidate version only; never overwrite the historical version being reproduced.

7 Operational evaluation and conclusion

7.1 Promotion gates

Gate	Pass condition
Data	Validation report complete; <=5% rejected; no schema drift; source checksum stored
Model	Recall >=0.95, ROC-AUC >=0.97, acceptable specificity, calibration no worse than champion
Slices	No material unexplained regression by approved site/device/cohort slices
Engineering	Unit/integration/DAG tests pass; vulnerability policy passes; signature and input example logged
Operations	Load test, dashboard, alert, rollback, and manual-only fallback verified
Governance	Model card updated; clinical/data/privacy owners approve promotion

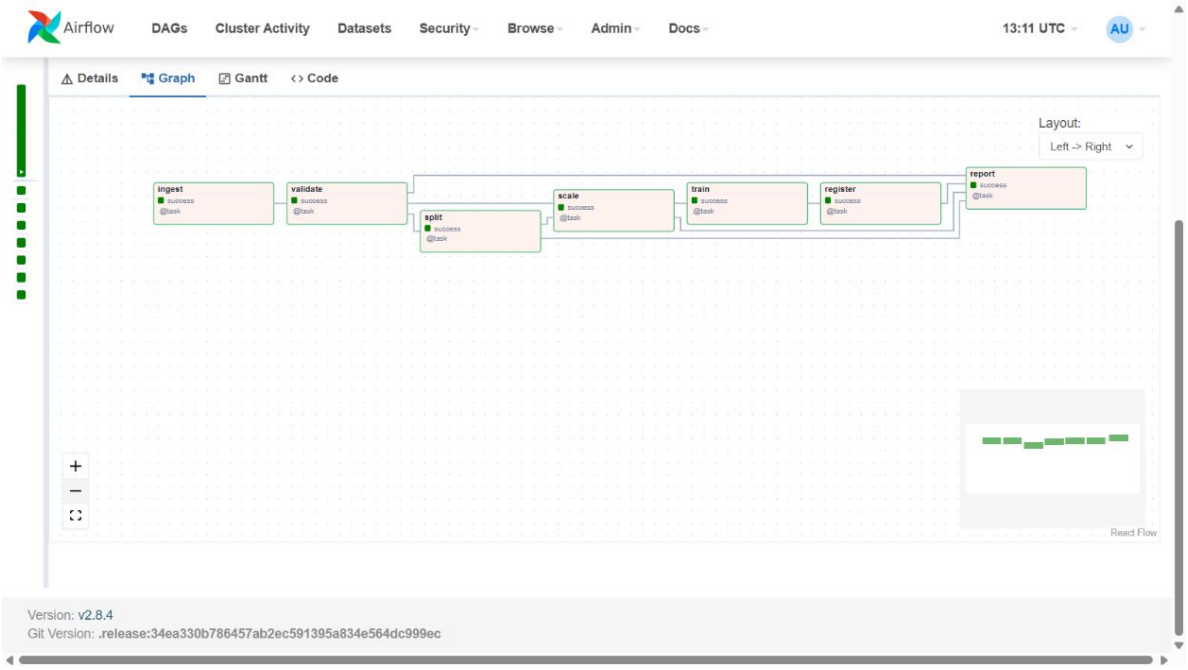
7.2 Deployment and monitoring sequence

- Register every successful candidate, but assign no production alias automatically.
- Run shadow scoring and compare champion/challenger outputs without changing clinician workflow.
- Canary the approved version to a small traffic slice; watch quality, latency, drift, and override signals.
- Expand only after a time-bounded review; immediately roll back or switch to manual-only on a stop condition.

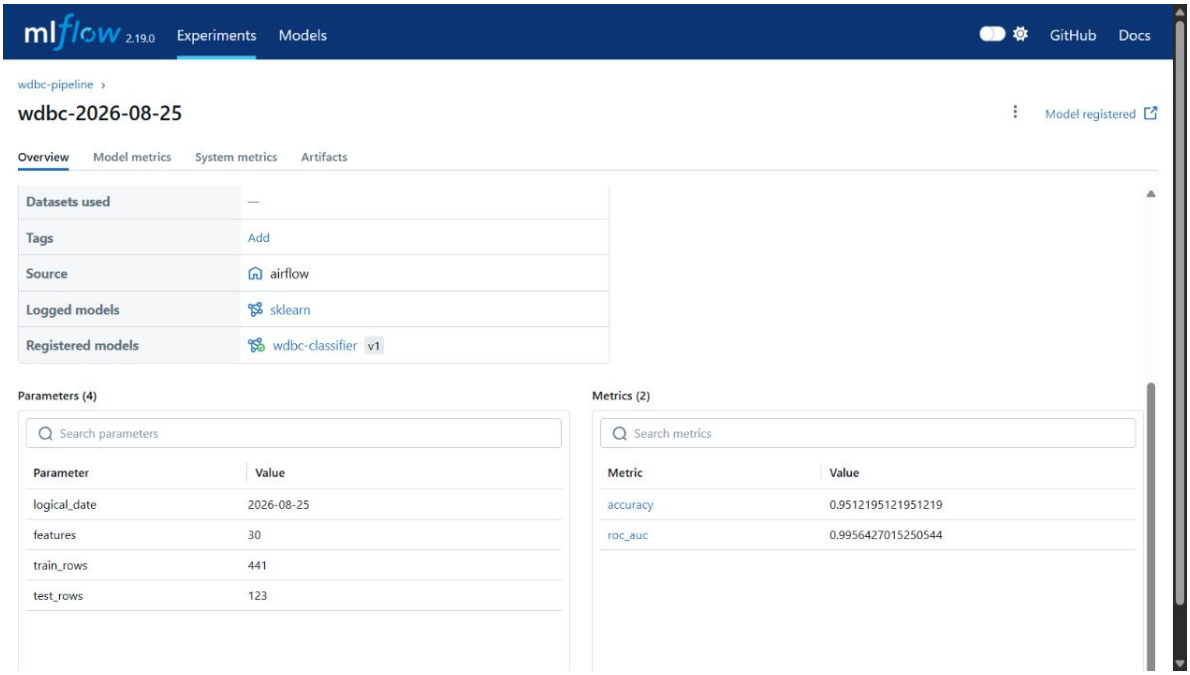
7.3 Conclusion

The pipeline demonstrates how MLOps converts a promising model into a controlled, inspectable, and repeatable system. Airflow provides explicit orchestration and failure semantics; MLflow provides run lineage, comparable evidence, and immutable model versions. The design keeps promotion and clinical use behind human review, which is the central operational requirement for this scenario.

Appendix A - Implementation evidence



Airflow graph view: seven successful tasks in the implemented DAG.



MLflow run view: parameters, metrics, and registered model link for the example run.

Appendix B - Selected implementation mapping

Report concept	Workspace evidence
Seven-stage orchestration	dags/wdbc_pipeline.py
Airflow and MLflow services	docker-compose.yml
Pinned runtime dependencies	Dockerfile and requirements.txt
Reproduced source data	data/raw/wdbc.csv
Run and registry evidence	docs/screenshots/airflow-*.png and mlflow-*.png
Operational instructions	README.md

References

- DDM501. Individual Assignment 2: ML Pipeline Design and MLOps Analysis. Course brief supplied with this submission.
- Apache Airflow documentation. https://airflow.apache.org/docs/
- MLflow documentation. https://mlflow.org/docs/latest/
- scikit-learn documentation. https://scikit-learn.org/stable/
- UCI Machine Learning Repository. Breast Cancer Wisconsin (Diagnostic) dataset.
- Workspace implementation and evidence, accessed September 2026.